

Desafio Técnico: Sistema de Gerenciamento de Planos de Aula

Objetivo: Desenvolver uma aplicação para gerenciamento de planos de aula. O sistema deve permitir o cadastro, a organização e a consulta de planos, além de utilizar Inteligência Artificial para auxiliar o usuário com recomendações de conteúdo para as aulas.

1. Cenário

A equipe de desenvolvimento do laboratório de pesquisa está criando uma plataforma centralizada para apoiar o planejamento de aulas e a organização de conteúdos pedagógicos. Para agilizar o trabalho dos docentes e conteudistas, o sistema não deve ser apenas um cadastro passivo, mas sim uma ferramenta que sugira conteúdos e tópicos complementares com base no tema da aula, utilizando uma API de LLM ou serviço equivalente.

2. Requisitos Funcionais

1. CRUD de Planos de Aula:

- Listagem com paginação.
- Cadastro, edição e exclusão.
- Campos: Título da Aula, Objetivo, Ementa/Resumo, Data Prevista, Disciplina, Conteúdos, Recursos de Apoio e Tags.

2. Funcionalidade “Smart Assist” (IA):

- No formulário de cadastro, deve haver um botão “Gerar Recomendações com IA”.
- Ao clicar, o frontend envia o Título da Aula, a Disciplina e a Ementa/Resumo para o backend.
- O backend consulta uma API de IA e retorna sugestões de conteúdos complementares, tópicos relacionados e 3 tags recomendadas.
- O frontend preenche automaticamente os campos com a resposta.

3. Organização e Consulta:

- O usuário deve conseguir filtrar planos de aula por Disciplina, Tags e Data Prevista.
- O usuário deve conseguir buscar planos por Título da Aula.
- A listagem deve permitir ordenação por Título ou data de cadastro.

3. Requisitos Técnicos

Backend

Escolha uma das stacks abaixo para construir a API RESTful:

Opção A - Python

- Framework: Flask.
- Uso de bibliotecas de validação de dados.

Opção B - JavaScript

- Frameworks: Node.js com Express ou Fastify.
- Uso de bibliotecas de validação de dados.

Requisitos comuns de Backend:

- Banco de Dados: MySQL, PostgreSQL ou SQLite.
- Integração com LLM: implementar a chamada à API escolhida.
- Criar um prompt claro para garantir que a IA responda no formato correto, preferencialmente JSON.
- Segurança: a chave da API não deve ficar hardcoded no código, devendo ser configurada por variáveis de ambiente.

Frontend

- SPA (Single Page Application).
- Feedback visual para o usuário enquanto a IA está “pensando” (loading state).
- Tratamento de erro caso a API de IA falhe ou demore a responder.
- Tela de listagem com filtros e paginação.
- Formulário de cadastro/edição com validação.

4. DevOps & Observabilidade

Item Bônus/Diferencial

- Integração Contínua (CI):
 - Pipeline no GitHub Actions, ou similar, que rode o linter a cada push.
 - Sugestão: flake8/black para Python ou ESLint/Prettier para JavaScript.
- Observabilidade:
 - Logs estruturados, especialmente nas operações principais e na interação com a IA.
 - Exemplo de log esperado:
[INFO] AI Request: Title="Introdução ao OSPF", Discipline="Redes", TokenUsage=180, Latency=1.4s.
 - Endpoint de Health Check (/health).
- Containerização:
 - A aplicação deve ser executada com Docker.
 - O projeto deve possuir, no mínimo, Dockerfile(s) e um arquivo docker-compose.yml para facilitar a execução local e em nuvem.
 - O ambiente deve permitir subir a aplicação com um único comando.

5. Entrega

- Repositório Git público.

- README.md detalhado.
- Suba em seu repositório um vídeo de até 5 minutos apresentando a solução desenvolvida, com enfoque nas principais escolhas técnicas, decisões de implementação, organização do projeto e eventuais dificuldades encontradas.
- Caso implemente itens bônus/diferenciais, eles devem ser demonstrados no vídeo ou descritos no README.

Dicas para o Candidato

- Atenção ao prompt engineering: instrua a IA para atuar como um “Assistente Pedagógico” para sugerir conteúdos úteis para a aula.
- Cuidado com o vazamento de chaves de API no GitHub. Use arquivos .env e inclua-os no .gitignore.
- Mostre que você domina a linguagem escolhida: use boas práticas de organização, validação, tratamento de erros e integração entre frontend e backend.
- Diferenciais como Docker, observabilidade, testes e deploy simples serão considerados diferenciais muito positivos, mas não são obrigatórios.